

IT-457: Lab Assignment 1

virajkumar patel / 202611047

August 13, 2026

Exercise 1: TCP Client-Server Q&A System

Problem Statement

Create a TCP server and client using the sockets library. Make them communicate with each other by making a Q&A system between them. Make a list of 4-5 Q&A pairs in a file and use these for the application.

Input Files

1. qa.txt

This file contains the predefined questions and answers separated by a pipe (|) symbol.

```
1 What is TCP?|TCP is a connection-oriented and reliable transport layer
  protocol.
2 What is UDP?|UDP is a connectionless and unreliable transport layer
  protocol.
3 What is a socket?|A socket is an endpoint used for communication
  between two processes.
4 What is IP?|IP stands for Internet Protocol and is used for addressing
  and routing packets.
5 What is HTTP?|HTTP stands for HyperText Transfer Protocol and is used
  for communication on the web.
```

2. TCP Server (server.py)

```
1 import socket
2 # Create a TCP socket
3 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4
5 # Bind the socket to localhost (127.0.0.1) and port 8080
6 server.bind(("127.0.0.1", 8080))
7
8 # Listen for incoming connections
9 server.listen(1)
10 print("TCP Server started...")
11 print("Waiting for client...")
12 # Accept an incoming connection from the client
13 client, address = server.accept()
14 print("Client connected!")
15
```

```

16 # Infinite loop to keep processing client's questions
17 while True:
18     # Receive the question from the client (up to 1024 bytes) and
    decode it
19     question = client.recv(1024).decode()
20
21     # If no data is received, break the loop
22     if not question:
23         break
24
25     # If client sends "exit", break the loop to close the connection
26     if question == "exit":
27         break
28
29     # Default answer if the question is not found in the file
30     answer = "Question not found."
31
32     # Open the Q&A file in read mode
33     file = open("qa.txt", "r")
34
35     # Read the file line by line
36     for line in file:
37         # Split the line into question and answer using the '|'
    delimiter
38         parts = line.strip().split("|", 1)
39
40         # Ensure the line has exactly two parts (Question and Answer)
41         if len(parts) == 2:
42             storedQuestion = parts[0]
43             storedAnswer = parts[1]
44
45             # If the received question matches the stored question, set
    the answer
46             if question == storedQuestion:
47                 answer = storedAnswer
48                 break # Stop searching once the match is found
49
50     # Close the file after reading
51     file.close()
52     print("Question:", question)
53     print("Answer:", answer)
54
55     # Encode and send the answer back to the client
56     client.send(answer.encode())
57
58 # Close the client connection and server socket
59 client.close()
60 server.close()

```

3. TCP Client (client.py)

```

1 import socket
2
3 # Create a TCP socket (AF_INET for IPv4, SOCK_STREAM for TCP)
4 client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6 # Connect to the server running on localhost (127.0.0.1) at port 8080

```

```

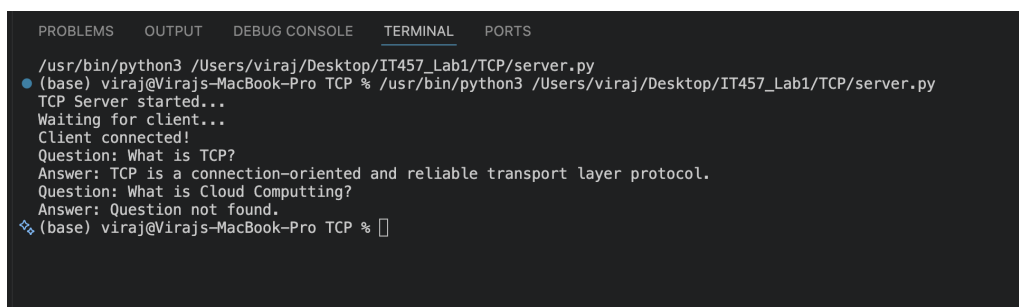
7 client.connect(("127.0.0.1", 8080))
8
9 print("Connected to server!")
10
11 # Infinite loop to keep asking questions to the server
12 while True:
13
14     # Take question input from the user via terminal
15     question = input("\nEnter question: ")
16
17     # Encode and send the question to the server
18     client.send(question.encode())
19
20     # If the user types "exit", break the loop to close the client
21     if question == "exit":
22         break
23
24     # Receive the answer from the server (up to 1024 bytes) and decode
    it
25     answer = client.recv(1024).decode()
26
27     # Print the received answer from the server
28     print("Server:", answer)
29
30 # Close the client socket connection
31 client.close()

```

Output and Explanation

Explanation of Execution: The TCP socket is established using `SOCK_STREAM`. The server binds to localhost on port 8080 and waits for a client. Upon receiving a question, the server iteratively reads `qa.txt`, splitting lines by `|`. If a match is found, the server replies with the corresponding answer; otherwise, it sends “Question not found.” The client displays the server’s response. Entering “exit” gracefully terminates the connection on both ends.

Expected Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• /usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/TCP/server.py
• (base) viraj@Virajs-MacBook-Pro TCP % /usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/TCP/server.py
TCP Server started...
Waiting for client...
Client connected!
Question: What is TCP?
Answer: TCP is a connection-oriented and reliable transport layer protocol.
Question: What is Cloud Computing?
Answer: Question not found.
❖ (base) viraj@Virajs-MacBook-Pro TCP %

```

Figure 1: TCP Server Console Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

/usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/TCP/client.py
• (base) viraj@Virajs-MacBook-Pro TCP % /usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/TCP/client.py
Connected to server!

Enter question: What is TCP?
Server: TCP is a connection-oriented and reliable transport layer protocol.

Enter question: What is Cloud Computing?
Server: Question not found.

Enter question: exit
❖ (base) viraj@Virajs-MacBook-Pro TCP %
```

Figure 2: TCP Client Console Output

Exercise 2: UDP Client-Server with Checksum Error Detection

Problem Statement

Create a UDP server and client using socket programming. Compute a 32-bit checksum over the data packet and include it in the packet header. The server and client must check for errors before accepting a packet; otherwise, respond with an error message. Test the code by artificially introducing an error.

Input Files

1. UDP Server (server.py)

```
1 import socket
2 import zlib
3 # Create a UDP socket
4 server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5
6 # Bind the socket to localhost (127.0.0.1) and port 8081
7 server.bind(("127.0.0.1", 8081))
8
9 print("UDP Server started...")
10 print("Waiting for packets...")
11
12 # Infinite loop to keep receiving packets
13 while True:
14     # Receive data and client address (buffer size 1024)
15     packet, address = server.recvfrom(1024)
16     packet = packet.decode()
17
18     # Split the received packet into message and checksum
19     parts = packet.split("|")
20
21     if len(parts) == 2:
22         message = parts[0]
23         received_checksum = int(parts[1])
24
25         # Calculate the 32-bit checksum (CRC32) of the received message
26         calculated_checksum = zlib.crc32(message.encode())
27
28         # Check for errors by comparing received checksum with
29         # calculated checksum
30         if received_checksum == calculated_checksum:
31             print(f"\n[SUCCESS] Valid message from {address}: {message}")
32             # Send success response to client
33             server.sendto(b"Packet accepted: No errors found.", address)
34         else:
35             print(f"\n[ERROR] Corrupted packet received from {address}!")
36             # Send error response to client
37             server.sendto(b"Error: Packet corrupted during transmission!", address)
```

2. UDP Client (client.py)

```
1 import socket
2 import zlib
3
4 # Create a UDP socket (AF_INET for IPv4, SOCK_DGRAM for UDP)
5 client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
6 serverAddress = ("127.0.0.1", 8081)
7
8 print("UDP Client started!\n")
9
10 while True:
11     # Take message input from the user
12     message = input("Enter message to send (or 'exit' to quit): ")
13
14     if message == "exit":
15         break
16
17     # Calculate CRC32 checksum for the original message
18     checksum = zlib.crc32(message.encode())
19
20     # Option to introduce an artificial error
21     choice = input("Do you want to send a corrupted packet to test
22 error handling? (y/n): ")
23
24     if choice.lower() == "y":
25         # Alter the message but keep the original checksum
26         packet = message + "x|" + str(checksum)
27         print("Artificial error introduced!")
28     else:
29         # Normal packet
30         packet = message + "|" + str(checksum)
31
32     # Send packet to server
33     client.sendto(packet.encode(), serverAddress)
34
35     # Receive the response from the server
36     response, address = client.recvfrom(1024)
37     print("Server Response:", response.decode(), "\n")
38
39 client.close()
```

Output and Explanation

Explanation of Execution: The UDP communication is implemented using `SOCK_DGRAM`. Because UDP provides unreliable datagrams, a 32-bit CRC32 checksum is calculated using Python's `zlib` library and appended to the message using the `|` delimiter. The user is given a prompt to optionally corrupt the payload. If an error is introduced, the client modifies the message string but sends the original checksum. Upon receiving the packet, the server recalculates the CRC32 checksum on the received string. If the recalculated checksum mismatches the received checksum, the server identifies the corruption and responds with an error.

Expected Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/UDP/server.py
(base) viraj@Virajs-MacBook-Pro UDP % /usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/UDP/server.py
UDP Server started...
Waiting for packets...

[SUCCESS] Valid message from ('127.0.0.1', 65380): "What is Cloude Computing?"

[ERROR] Corrupted packet received from ('127.0.0.1', 65380)!
█
```

Figure 3: UDP Server Console Output showing success and error handling

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
/usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/UDP/client.py
(base) viraj@Virajs-MacBook-Pro UDP % /usr/bin/python3 /Users/viraj/Desktop/IT457_Lab1/UDP/client.py
UDP Client started!

Enter message to send (or 'exit' to quit): "What is Cloude Computing?"
Do you want to send a corrupted packet to test error handling? (y/n): n
Server Response: Packet accepted: No errors found.

Enter message to send (or 'exit' to quit): What is Network Computing?
Do you want to send a corrupted packet to test error handling? (y/n): y
Artificial error introduced!
Server Response: Error: Packet corrupted during transmission!

Enter message to send (or 'exit' to quit): exit
(base) viraj@Virajs-MacBook-Pro UDP % █
```

Figure 4: UDP Client Console Output demonstrating artificial error introduction